

CSC-304 Parallel and Distributed Computing
CCP Report
Section: A



Submitted By:

Syed Shayan Arshad	2023-CS-625
Adeel-Ur-Rehman	2023-CS-626
Hammad Khalid	2023-CS-637
Waqar Haider	2023-CS-640

Submitted To:

Sir Nadeem Iqbal

Dated: 7th June 2026

Department of Computer Science
University of Engineering and Technology Lahore, New Campus

Distributed Matrix Multiplication Using Python Socket Programming

1. Introduction

Problem Description

Matrix multiplication is a fundamental operation used in many fields such as scientific computing, machine learning, image processing, and data analysis. When the size of matrices becomes very large, the computation requires significant processing time and resources. A single machine may not be able to perform these calculations efficiently, especially for large-scale applications.

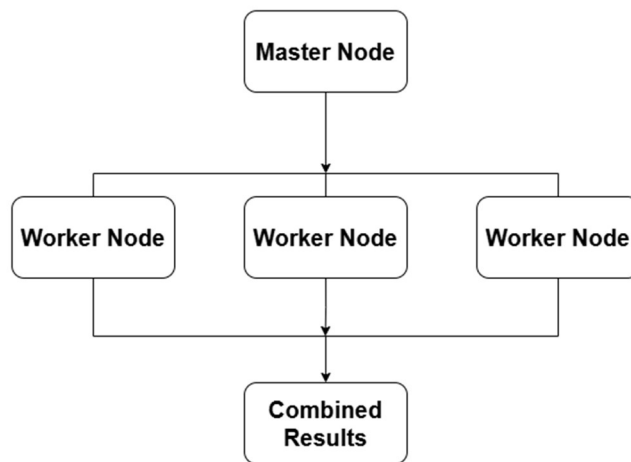
To address this issue, this project implements distributed matrix multiplication, where the workload is divided among multiple worker nodes. Each worker performs a portion of the computation, and the results are combined to generate the final output matrix. The objective of this project is to reduce computation time and evaluate the performance improvements achieved through parallel processing.

Selected Communication Technology

The distributed system was implemented using Python Socket Programming with the TCP protocol. TCP sockets were selected because they provide reliable communication between the master node and worker nodes. NumPy was used for matrix generation and multiplication operations because of its optimized numerical computation capabilities.

2. System Design

Architecture Diagram



Task Distribution Strategy

The master node generates matrices A and B. Matrix A is divided into row-wise chunks according to the number of available worker nodes. Each worker receives:

- A portion of matrix A
- The complete matrix B

The worker performs matrix multiplication on its assigned rows and computes a partial result. After computation, the worker sends the result back to the master node. The master collects all partial results and combines them in the correct order to form the final output matrix.

3. Experimental Results

Single-Machine Execution Time

Matrix Size	Execution Time (s)
1000 × 1000	1.655
2000 × 2000	35.335
4000 × 4000	384.567

Distributed Execution Time

1000 × 1000 Matrix (7.63 MB)

Workers	Worker 1	Worker 2	Worker 3	Max Compute (s)	Speedup	Network (s)
2 Systems	0.626	1.453	-	1.453	1.14x	10.908
3 Systems	0.517	0.337	0.727	0.727	2.29x	9.9

2000 × 2000 Matrix (30.52 MB)

Workers	Worker 1	Worker 2	Worker 3	Max Compute (s)	Speedup	Network (s)
2 Systems	16.52	19.071	-	19.071	1.85x	46.862
3 Systems	11.380	6.758	29.739	29.739	1.18x	59.418

4000 × 4000 Matrix (122.07 MB)

Workers	Worker 1	Worker 2	Worker 3	Max Compute (s)	Speedup	Network (s)
2 Systems	178.885	302.606	-	302.606	1.27x	789.396
3 Systems	135.638	164.904	335.549	335.549	1.14x	433.811

4. Performance Comparison

Speedup Analysis

The distributed implementation reduced computation time compared to the single-machine execution in all test cases. The best performance was achieved for the **1000 × 1000 matrix**, where three worker nodes provided a speedup of **2.29x**.

For the 2000×2000 and 4000×4000 matrices, two worker nodes achieved higher speedups (**1.85x** and **1.27x**) than three worker nodes (**1.18x** and **1.14x**). This occurred because the third worker machine had a slower processor, causing it to take significantly longer to complete its assigned workload.

Observations and Discussion

Several important observations were made during the experiments:

1. Distributed processing improved performance by executing matrix multiplication in parallel.
2. The highest speedup (**2.29x**) was achieved using three workers for the 1000×1000 matrix.
3. For larger matrices, the slower third worker became a bottleneck and increased the total execution time.
4. Network communication overhead increased as matrix size grew, especially for the 4000×4000 matrix.
5. The results show that the performance of a distributed system depends not only on the number of workers but also on the hardware capability of each worker node.

5. Conclusion

Key Findings

This project successfully implemented distributed matrix multiplication using Python socket programming. The workload was divided among multiple worker nodes, allowing matrix multiplication to be performed in parallel.

The results showed that distributed execution can reduce computation time and improve performance compared to a single-machine implementation. The highest speedup achieved was **2.29x** using three worker nodes for the 1000×1000 matrix. However, for larger matrices, the performance improvement was limited because one worker node had a slower processor and required more time to complete its assigned task.

Challenges Faced

The following challenges were encountered during the implementation:

- Efficiently transferring large matrix data across the network.
- Managing increasing communication overhead for larger matrices.
- Achieving balanced workload distribution among worker nodes.
- Preventing slower worker nodes from becoming bottlenecks that affect overall system performance.

Overall, the project demonstrated the practical advantages and limitations of distributed matrix multiplication. The results highlight the importance of balancing computation and communication costs when designing distributed computing systems.